

Dokumentacja Projektu „ARENA”

Etap 1 - Architektura aplikacji

Mikołaj Ratajczak

26 maja 2026

1 Wstęp

Dokument zawiera uproszczony opis klas i metod, których znajomość jest nie zbędna do wykonania drugiego etapu projektu. Część metod oraz przygotowanych klas została pominięta.

2 Metody potrzebne przy realizacji klasy Settings

2.1 Konstruktor klasy Button

Poniżej przedstawiono uproszczony konstruktor Klasy button:

```
1 Button(std::string text, sf::Font& font, sf::Vector2f pos, sf::Vector2f siz)
```

Gdzie:

std::string text - jest tekstem wyświetlanym na przycisku (jego rozmiar jest automatycznie ustawiony na nieco mniej niż połowę ustawionej wysokości przycisku. Jeżeli tekst nie mieścił by się w przycisku należy zwiększyć wartość 'x' zostawiając poprzednią wartość 'y').

sf::Font& font jest referencją do czcionki przechowywanej w **TextureMenager** w klasie **Player** oraz **Enemy** LUB w przypadku klas **Settings**, **Shop** oraz **Arena**, jest on przechowywany w **struct Menagers**.

sf::Vector2f pos - zawiera pozycje 'x' oraz 'y', w której będzie znajdował się GEOMETRYCZNY ŚRODEK przycisku.

sf::Vector2f siz - zawiera wysokość i szerokość tworzonego przycisku.

PRZYKŁADOWE UŻYCIE:

```
1 Button("Swords", menagers.tex.getMainFont(), sf::Vector2f(50, 300), sf::Vector2f(120, 50))
```

2.2 Metody klasy Button

Poniżej przedstawiono najważniejsze metody klasy Button:

```
1 bool IsClicked(Utils::Mouse& mouse)
```

Metoda odpowiada za animację przycisku. Jeżeli do utworzenia przycisku wykorzystano domyślny konstruktor - kolory normalnego oraz najechanego przycisku są ustalane jako domyślne - czyli zdefiniowane w pliku 'theme.h' w namespace Theme.

Funkcja zwraca true gdy najechany jest kursor i kliknięty lewy przycisk myszy. False gdy nie kliknięty.

```
1 void Draw(sf::RenderWindow& window)
```

Metoda odpowiada za rysowanie przycisku w SFML. Należy ją umieścić w już utworzonej metodzie `Settings::Draw(sf::RenderWindow& window)` i wywołać dla każdego utworzonego przycisku. W SFML 'maluje' się jak na płótnie - czyli zaczynamy od spodu a na końcu to co ma być na samym wierzchu.

Poniżej przedstawiono dodatkowe metody klasy Button:

```
1 void ChangePosition(float x, float y)
```

Zmienia pozycję ŚRODKA GEOMETRYCZNEGO wcześniej utworzonego przycisku.

```
1 void ChangeSize(float x, float y)
```

Zmienia rozmiar już utworzonego przycisku.

```
1 void ChangeTextColor(sf::Color color)
```

Zmienia kolor text w utworzonym przycisku.

2.3 Metody w pliku 'utils.h' - dodatkowe przydatne funkcje

```
1 void CenterTextOrigin(sf::Text& text)
```

Funkcja przyjmuje referencje na już utworzony obiekt `sf::Text` (czyli sam text wyświetlany w SFML) i ustawia jego punkt zaczepienia w miejscu obliczonego środka geometrycznego napisu. Należy pamiętać, że metoda ta zmienia również kolor tekstu na domyślny - zdefiniowany w pliku `utils.h`.

```
1 void CenterSpriteOrigin(sf::Sprite& sprite)
```

Funkcja przyjmuje referencje na już utworzony `sf::Sprite` (czyli 'duszka' z załadowaną teksturą).

2.4 Klasa MessageMenager - komunikaty

```
1 void add(std::string text, MessageType mtype, float tim)
```

Metoda dodająca wiadomość, która zostanie wyświetlona w postaci powiadomienia, gdzie:

`std::string text` - to wyświetlany tekst powiadomienia.

`MessageType mtype` - to typ z `enum class MessageTypeError, Warning, Success`, który przypisuje odpowiedni kolor wiadomości.

`float tim` - to czas wyświetlania wiadomości, wyrażony w sekundach.

RYSOWANIE ORAZ UPDATOWANIE WIADOMOŚCI NIE WYMAGA WASZEJ INGERENCJI

- jest automatyczne.

2.5 Wymagania projektowe dla klasy Settings

Klasa Settings ma za zadanie wczytać z pliku dowolnego formatu następujące które za pomocą NOWO UTWORZONYCH metod umieszczonych w klasie Player nadpiszą następujące informacje:

- `std::map<SwordsTypes, bool> unlockedSwords` - odblokowane miecze
- `std::map<ArmorsTypes, bool> unlockedArmors` - odblokowane zbroje
- `int gold` - posiadane złoto
- `int points` - posiadane punkty
- `std::string name` - nazwa gracza // NA RAZIE JEST ONA STAŁA WIĘC MOŻNA JĄ ZMIENIAĆ LUB NIE - wedle uznania ale wiązało by się to z stworzeniem interfejsu zmieniającego imię gracza.

ORAZ

- `SwordsTypes sword` - miecz, który miał założony gracz w chwili wykonania zapisu
- `ArmorsTypes armor` - zbroja, którą miał założoną gracz w chwili wykonania zapisu

Klasa Player posiada już metodę, która wyposaży go w miecz na podstawie pola 'sword' oraz 'armor'.

2.6 Rady projektowe dla klasy Settings

- Zapoznać się z konstrukcją klasy Shop, klasa Settings jest klasą z zasady bliźniaczą.
- Zalecam rozważyć powielenie sposobu przechowywania tekstu i przycisków z klasy shop.

3 Metody potrzebne przy realizacji metod walki

3.1 Pola klasy Player

Klasa posiada struct `PlayerStats stats` - przechowuje on następujące dane:

-`int health` - zdrowie gracza, które jest zmieniane wraz ze zmianą zbroji (przyjmuje wtedy wartość maksymalną - stąd nie wskazane jest wywoływanie metod `Player::setArmor()` oraz `Player::setSword()` w trakcie pobytu gracza na arenie.

-`int damage` - posiada informację o obecnie zadawanym demage gracza, wartość zmienia się automatycznie przy zmianie miecza.

Przykładowa zmiana statystyk:

`stats.health -= 10;` - odjęcie 10 pkt HP.

`stats.damage += 10;` - dodanie 10 pkt Ataku

Jeżeli stworzycie metody `getDamage` oraz `Hit()` w klasie `Player` nie będziecie potrzebować getterów i seterów, nie wiem czy chcecie zmieniać te statystyki w trakcie gry.

3.2 Metody klasy `Player`

Zmiana pozycji gracza odbywa się za pomocą metody:

- `Player::Update(float x, float y)` - przemieszcza ona środek geometryczny gracza w miejsce podanego punktu.

- `Player::Update()` - jest wywoływane przez funkcję wyżej ale nie zmienia pozycji gracza tylko, wywołuje update jego paska zdrowia.

- `Player::HpBarUpdate()` - aktualizuje 'pasek' zdrowia gracza - metoda jest automatycznie wywoływana w metodzie `Player::Update()`.

- `Player::Draw(RenderWindow& window)` - rysuje gracza i jego miecz - JEST JUŻ WYWOŁYWANE PRZEZ METODĘ `Draw()` klasy `ARENA` - Nie jest wymagana wasza ingerencja.

3.3 Pola klasy `Enemy`

Klasa podobnie jak `Player` posiada structure zawierającą statystyki, tu nazywa się ona `EnemyStats` i przechowuje ona następujące dane: - `int health` - poziom zdrowia przeciwnika, jest on automatycznie ustawiany w konstruktorze na wartość z pliku 'stats.h' dla danego przeciwnika. - `int damage` - zadawane obrażenia, podobnie jak poziom zdrowia jest wczytywany z pliku 'stats.h' dla danego przeciwnika.

3.4 Metody klasy `Enemy`

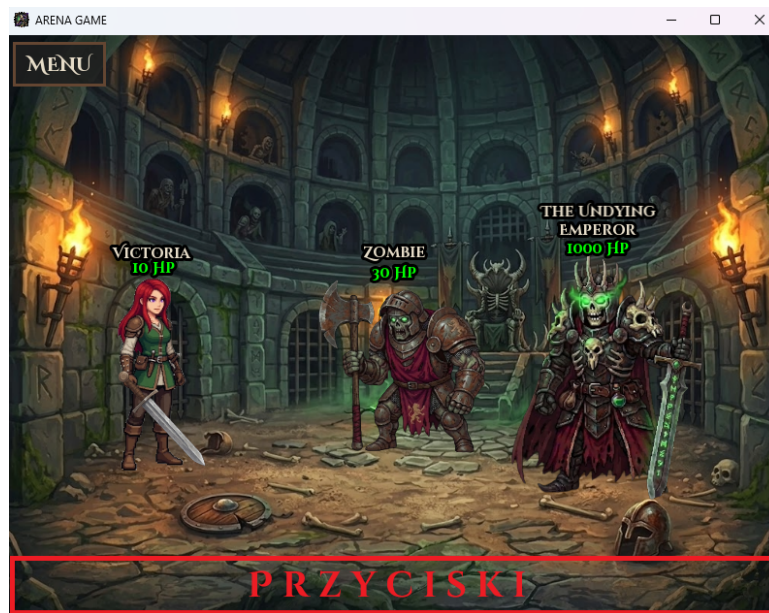
- Metody `Enemy::Update()` oraz `Enemy::Update(float x, float y)` działają analogicznie jak w klasie `Player`.

- Metoda `Enemy::Draw(sf::RenderWindow& window)` działa podobnie jak dla klasy `Player` i musi być wywołana dla każdego przeciwnika znajdującego się na arenie z osobna.

- Metoda `Enemy::HpBarUpdate()` jest wywoływane w metodzie `Update()`.

3.5 Wymagania projektowe (Jak ja to widzę - jeżeli macie inną wizję - zróbcie jak wam wygodniej)

- Stworzyć na samym dole areny pasek przycisków, wywołujący dane akcje w walce - Najpierw gracz klika lewym przyciskiem myszy na przeciwnika a następnie wybiera co chce zrobić (zakładam, że chcecie zrobić jakiś średnio-zaawansowaną mechanikę walki i będzie kilka opcji). Poniżej zamieszczono sugerowane miejsce paska z przyciskami



Rysunek 1: Sugerowane umieszczenie interfejsu walki

- Zaimplementować metody walki np. Hit, TakeDamage.
- Zbalansować statystyki poszczególnych itemów oraz przeciwników.
- Główna pętla ARENY jest w metodzie Arena::Fight() - w niej przewidziano wywoływanie mechanik walki.
- OPCJONALNIE: możecie dodać levele bazujące na ilości punktów zgromadzonych przez gracza i w zależności od tego vector przeciwników ma inne postacie.
- OPCJONALNIE: można pobawić się w animacje - dostarczę wam grafiki jeżeli będziecie chcieli.

3.6 Rady projektowe dla metod walki

- Zastosujcie tury: kolej gracza, przeciwnika.